

LLD Case Study 3: Movie Ticket Booking System (BookMyShow / Fandango)

Target Patterns: Concurrency Control, State Machine, Factory Pattern, Strategy Pattern

Key Engineering Focus: High-concurrency seat locking, race condition elimination, temporary seat reservations with TTL expiration, and payment processing.

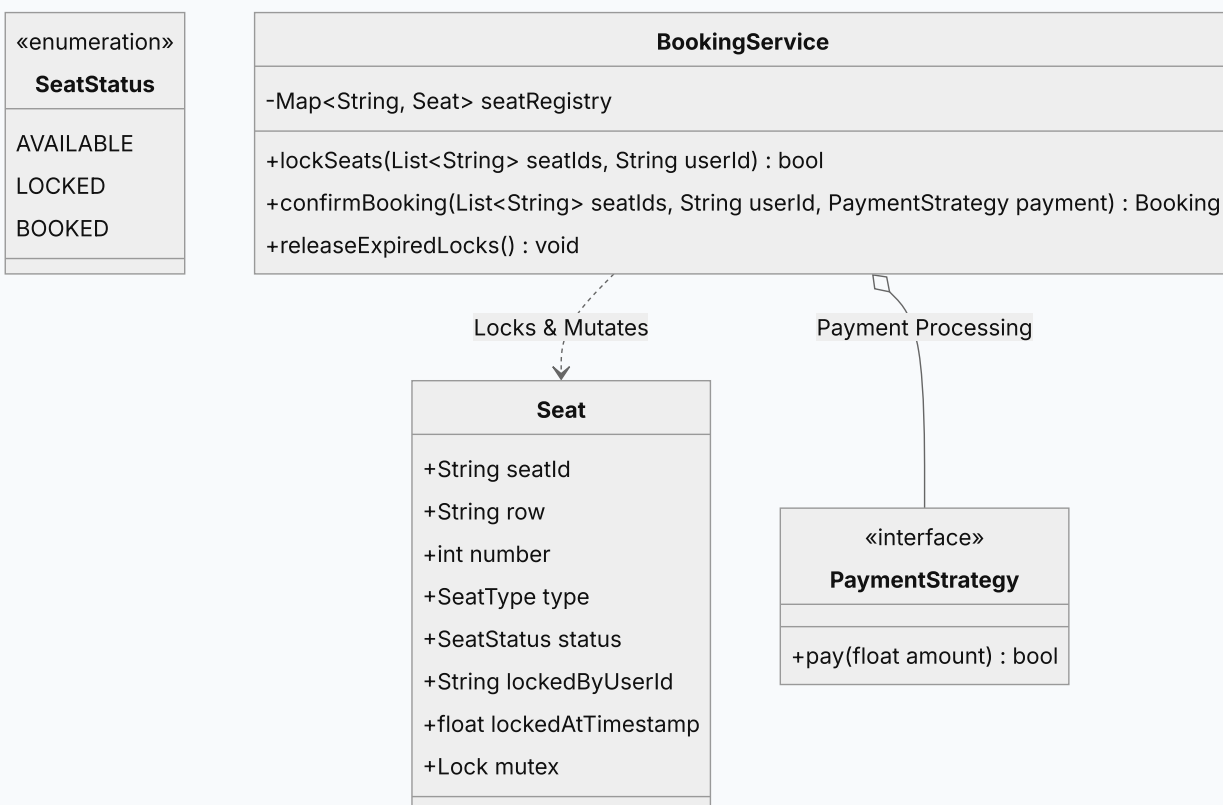
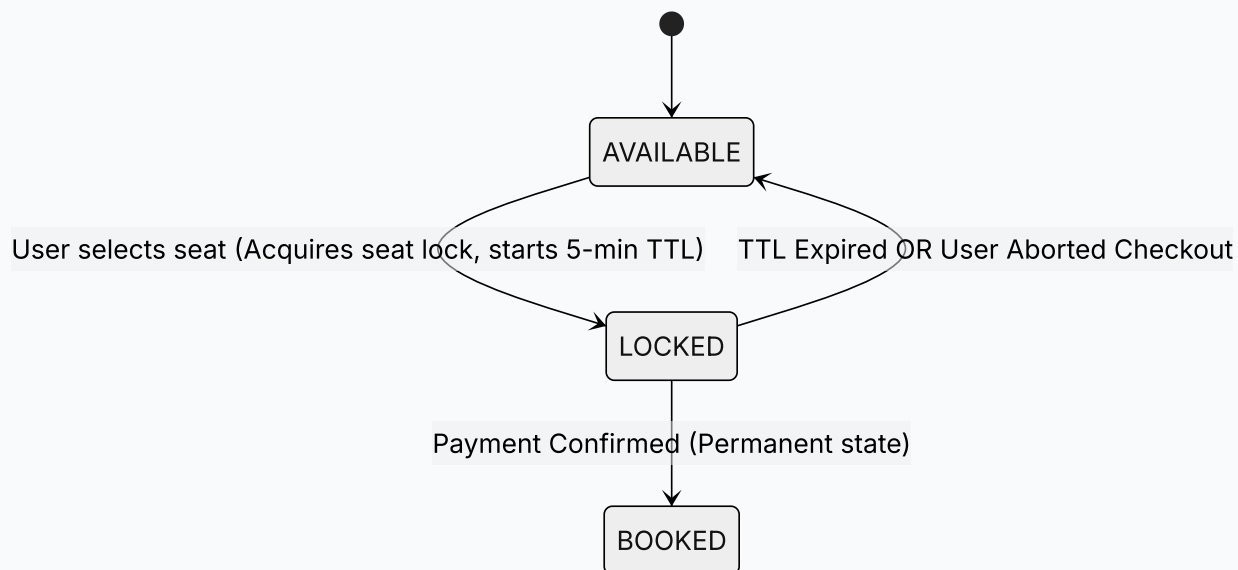
1. Problem Statement & Functional Requirements

Design the core booking engine for a cinema platform. Thousands of concurrent users vie for high-demand premiere seats.

Requirements:

1. **Cinema Hierarchy:** Cinema → Audi / Screen → Show → Seats (Regular, VIP, Recliner).
 2. **State Machine:** A seat can be `AVAILABLE`, `TEMPORARILY_LOCKED` (e.g., 5-minute payment hold window), or `BOOKED`.
 3. **Concurrency Control:** Absolutely no two users can book or lock the same seat simultaneously. Handled via thread-safe atomic mutexes.
 4. **Auto-Expiration:** If payment is not completed within the hold window, the lock automatically expires and the seat returns to `AVAILABLE`.
 5. **Payment Processing:** Decoupled via Strategy Pattern (UPI, CreditCard, NetBanking).
-

2. Architecture & State Diagram



3. Production-Grade Python Implementation

